BA01014@umbc.edu

Lab 2

To: Nafisa Tasnim

From: Isaac Boteler

Date: February 27th, 2026

Subject: Hamming Distance

BA01014@umbc.edu

Objective:

To write and test an assembly program to find the hamming distance from the binary representation of two strings.

Approach:

To calculate the hamming distance we need to sum all the bit differences from two binary inputs. To do so, we used the xor instruction to create a single stream in which we could sum all the ones to find the hamming distance. Then we could check the first bit, increment if 1 then bit shift to the right until the stream became empty. Due to the size of the inputs this had to be an iterative process. We decided to load the inputs a byte at a time since string characters are stored as a byte. The process would then end when either of the input bytes loaded as a null.

A flow chart of our algorithm is as follows:

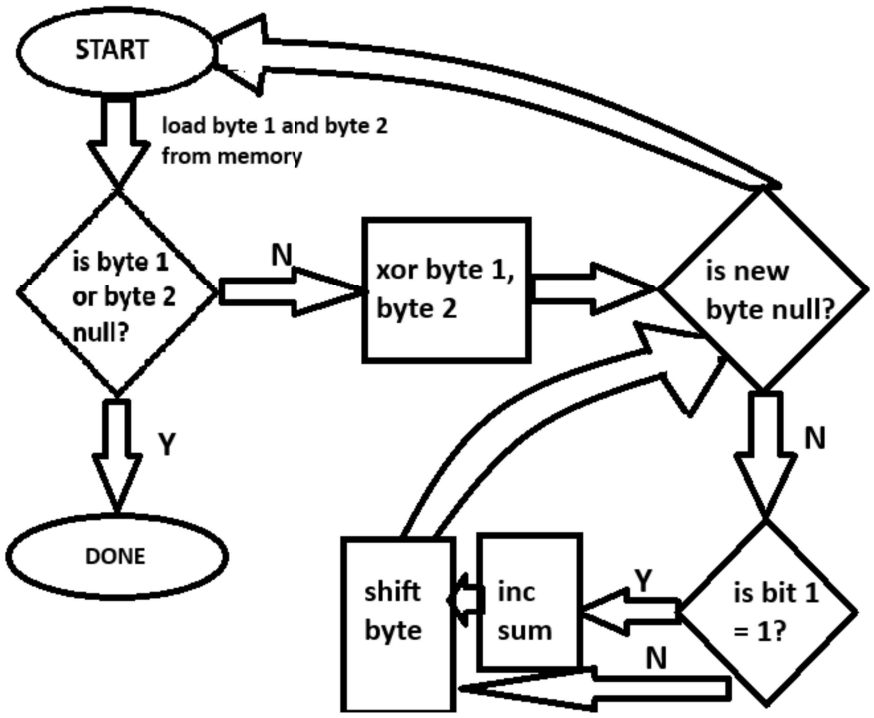

Figure 1: flowchart of hamming algorithm

The decision to load a byte, instead of 8 bytes was made so that we could check for the null character more cheaply. The alternative might have been faster, it would reduce the number of load instructions but our method is simpler to understand and debug.

Drawing from our first lab we used a C program as our driver to handle both the inputs and the outputs. Our test program is also a C program that checks the function against a variety of normal and edge cases. The hamming distances for the program were computed by hand and checked against this website (after converting the text to binary that is).

There is little to no error handling in the assembly function. The program will continue to load data until it receives a null character from either of the strings. Input validation is handled in the driver code, we use fget() with a 256 character buffer. An issue with fget() is the end LF character. This character changed the behavior of strings with different lengths (ie: “a”, “aa” would output 5 because of the LF character after “a”). Because of this unsatisfactory outcome, we decided to strip the LF character from all inputs to prevent further unexpected results.

Code:

hamming_distance_calculator.s

```
# Isaac Boteler BA01014@umbc.edu

# CMPE 310 LAB 2 HAMMING DISTANCE
# find_hamming_distance function to calculate hamming distance between 2 strings
  .section    .bss
  .globl  string_1
  .globl  string_2

  .lcomm  string_1,       256                 # Reserve
  .lcomm  string_2,       256
  .section    .text

  .globl  find_hamming_distance               # Make function visible to C program

find_hamming_distance:
  mov     $0, %rbx                            # this is our index
```

```
    mov     $0, %rax                                # this is our aggregate
hamming_loop:
    movb    string_1(%rbx), %cl
    cmpb    $0, %cl                                 #check if str1 is null
    jz      done
    movb    string_2(%rbx), %cl
    cmpb    $0, %cl                                 #check if str2 is null
    jz      done

    movb    string_2(%rbx), %cl                     # load first char
    xor     string_1(%rbx), %cl                     # difference between chars

    inc     %rbx                                    # when byte_check is done, we load next
addr
byte_check:                                        # cl register is read by bit, shifted
left until null
    test    %cl,    %cl                             # check for null
    jz      hamming_loop

    mov     $1,     %ch                             # check bit for cl
    and     %cl,    %ch                             # prepare for check
    shr     %cl                                     # load next bit (shift byte right)
    test    $1,     %ch                             # check for matching bit
    jz      byte_check                              # bits are same, move on

    inc     %rax                                    # bits are not same, increase ham
distance
    jmp     byte_check


done:
    ret

.section .note.GNU-stack,"",@progbits
```

hamming_distance_driver.c

```
//Isaac Boteler BA01014@umbc.edu

//CMPE 310 LAB 2
//Driver code for hamming_distance_calculator
//Prompts user for 2 strings, returns hamming distance
//Loops until user requests exist
//string input must be 255 characters or less
//string must be of standard printable characters (ASCII 33 - 127)
```

BA01014@umbc.edu

```c
#include <string.h>
#include <stdio.h>

//for strings up to size 255 we need an additional character for null
const int INPUT_SIZE = 256;
// inputs for assembly function
extern unsigned char string_1[];
extern unsigned char string_2[];
extern int find_hamming_distance(void);   // Assembly function

void getInput(char* input); //function for getting input

int main()
{
  //Welcome
  printf("Welcome to the program to calculate the hamming distance between 2 strings\n");
  //Prompt to exit
  char user_input = 'n';
  //main loop
  while (user_input != 'y' && user_input != 'Y') {
    //Prompt for string
    printf("please input the first string \n");
    getInput(string_1);
    printf("Please input the second string \n");
    getInput(string_2);
    printf("Thank you, the hamming distance for strings ");
    printf(string_1);
    printf(" and ");
    printf(string_2);
    printf("\nis: ");

    //Function call to assembly code
    int hamming_distance = find_hamming_distance();

    printf("%d", hamming_distance);
    printf("\n");

    //Exit program prompt
    //Any character aside from y/Y will continue the program
    printf("exit program? (y/Y or n/N)\n");
    scanf("%c", &user_input);
    //flush newline
    while ((getchar()) != '\n');
```

```
    }


    return 0;
}
void getInput(char* input) {
    //prompt user until we have valid input
    //invalid input is input that is too long
    while (fgets(input, INPUT_SIZE, stdin) == NULL) {
        printf("input can be no longer than 255 characters \n");
    }
    //strip LF character from input, replace with null character
    input[strcspn(input,"\n")] = '\0';

    return;
}
```

hamming_distance_tester.c

```
//Isaac Boteler BA01014@umbc.edu

//CMPE 310 LAB 2 HAMMING DISTANCE TESTER
//Runs a series of tests against the find_hamming_distance function
//cases include normal and edge
#include <stdbool.h>
#include <stdio.h>
//Constants
const int INPUT_SIZE = 256;
const int NUM_TESTS = 10;

// inputs for assembly function
extern unsigned char string_1[];
extern unsigned char string_2[];
extern int find_hamming_distance(void);    // Assembly function

//Testing function
bool test_hamming(char *input_1, char* input_2,int distance);

int main()
{    printf("Testing the function find_hamming_distance\n");
    bool allTestsPassed = true;
    //tests are for normal and edge cases, the driver handles the error handling of
weird inputs
    //distance is calculated by hand and by
https://www.compscilib.com/calculate/hamming-distance?variation=default
    printf("Testing trivial:\n");
    if (allTestsPassed = test_hamming("1","1",0))
```

BA01014@umbc.edu

```
        printf("  Passed\n");
    else
        printf("  Failed\n");

    printf("Testing different string lengths (1):\n");
    if (allTestsPassed =
test_hamming("aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaa"
        ,"a",0))
        printf("  Passed\n");
    else
        printf("  Failed\n");

    printf("Testing different string lengths (2):\n");
    if (allTestsPassed =
test_hamming("aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaa"
        ,"b",2))
        printf("  Passed\n");
    else
        printf("  Failed\n");

    printf("Testing normal:\n");
    if (allTestsPassed = test_hamming("car","truck",10))
        printf("  Passed\n");
    else
        printf("  Failed\n");

    printf("Testing max length:\n");
    if (allTestsPassed =
test_hamming("aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaa"

,"bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb"
        ,510))
        printf("  Passed\n");
    else
```

```
    printf("  Failed\n");

  if (allTestsPassed)
    printf(" All Tests Passed :)\n");
  else
    printf(" Do Better :(\n");

  return 0;
}

bool test_hamming(char *input_1, char* input_2,int actual_distance) {
  //load inputs into function
  for (int i = 0;i < INPUT_SIZE; i++) {
    string_1[i] = input_1[i];
    //check for null (end of string)
    if (input_1[i] == '\0') {
      break;
    }
  }
  for (int i = 0;i < INPUT_SIZE; i++) {
    string_2[i] = input_2[i];
    //check for null (end of string)
    if (input_2[i] == '\0') {
      break;
    }
  }
  //calculate and check
  int calculated_distance = find_hamming_distance();
  //print inputs and outputs if test fails
  if (actual_distance != calculated_distance) {
    printf("%s \n%s \n", input_1, input_2);
    printf("Should have calculated: %d,Instead
%d",actual_distance,calculated_distance);
    return false;
  }
  return true;
}
```

Makefile

```
lab_2: hamming_distance_calculator.s hamming_distance_driver.c

  gcc -no-pie hamming_distance_calculator.s hamming_distance_driver.c -o lab_2

test: hamming_distance_calculator.s hamming_distance_tester.c
  gcc -no-pie hamming_distance_calculator.s hamming_distance_tester.c -o lab_2_test
```

BA01014@umbc.edu

```
debug: hamming_distance_calculator.s hamming_distance_driver.c
	gcc -g -no-pie hamming_distance_calculator.s hamming_distance_driver.c -o lab_2_debug
```

Output:

lab_2 sample output

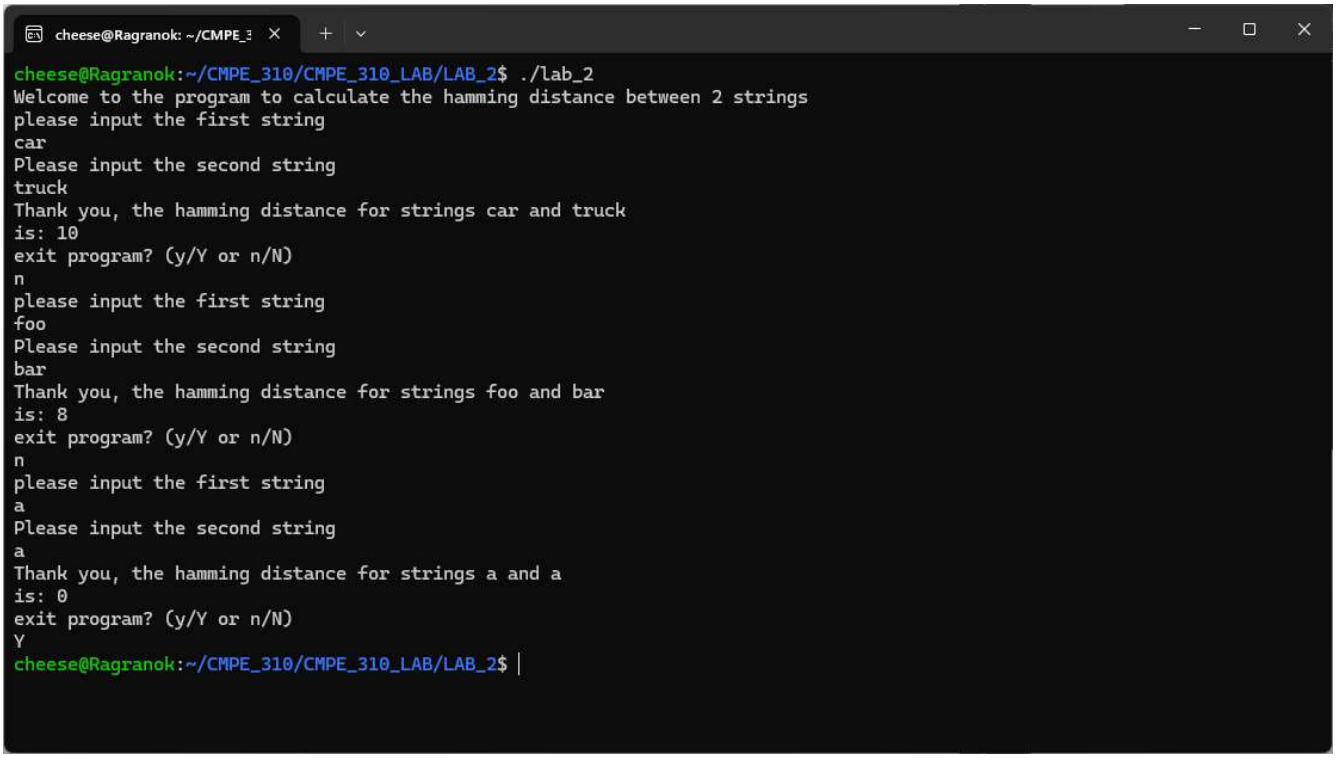

Figure 2: screenshot of terminal after running ./lab_2 with the following inputs: car, truck, n, foo, bar, n, a, a, Y

BA01014@umbc.edu

lab_2_test output

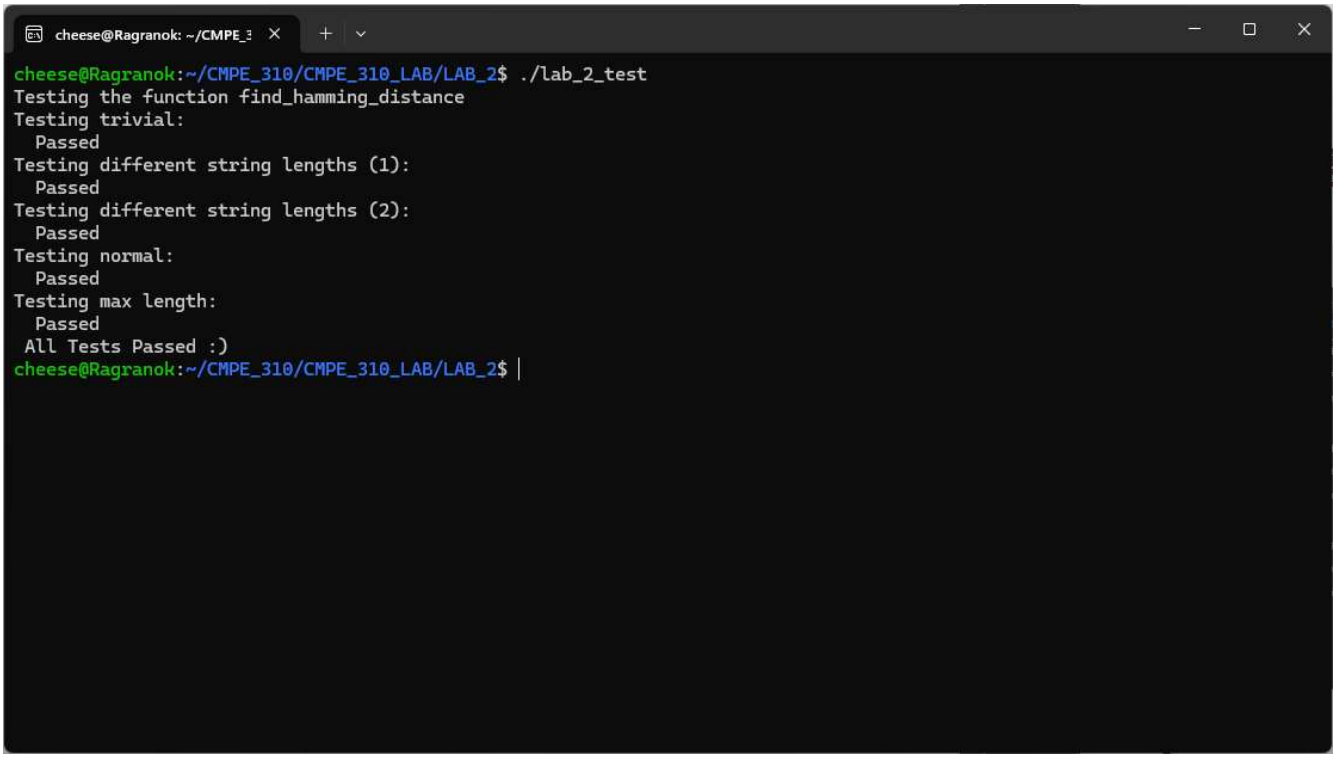

Figure 3: Screenshot of terminal after running ./lab_2_test

Repository:

All files can be found at the following link:

https://github.com/Gh21st/CMPE_310_LAB/tree/main/LAB_2